LITERATURE SUMMARY

Retrieval-Guided Repair and the Security of Vibe-Coded Apps

How retrieved fix patterns improve program repair, and how insecure real vibe-coded applications are

RAP-Gen (Wang et al., ESEC/FSE 2023) · Understanding the (In)Security of Vibe-Coded Applications (Deng, Fan, Meng, arXiv 2026)

OVERVIEW

Two papers at a glance

PAPER 01 · SALESFORCE AI RESEARCH · FSE 2023

RAP-Gen: Retrieval-Augmented Patch Generation

Problem: neural repair models must memorise every fix pattern in a fixed set of parameters, which caps how well they repair bugs.

Idea: retrieve the most similar past bug-fix pair from a codebase and give it to CodeT5 as a hint while it writes the patch.

PAPER 02 · ARXIV 2606.23130 · SEP 2026

Understanding the (In)Security of Vibe-Coded Applications

Problem: vibe-coded apps are built and deployed by AI agents with little human review, and no study had measured their real security.

Idea: collect real vibe-coded apps (VibeApps), audit 200 deployed ones (VibeVulns), find why agents introduce vulnerabilities and what reduces them.

Common thread: both papers are about **what to trust in AI-written code**. RAP-Gen shows that a good external example improves a fix; the security study shows what goes wrong when nobody checks the result.

Summaries compiled from the two papers' reported results.

Retrieve a past fix, then generate the patch

STEP 1 · HYBRID RETRIEVER

Find the closest bug-fix pair

BM25 scores word overlap (lexical). A **dense retriever** (DPR built on the CodeT5 encoder) scores meaning. The two scores are added (weight 1) and the top-1 pair is kept.

→

STEP 2 · AUGMENT THE INPUT

Buggy code + hint

Input becomes: *error info, buggy code, [BUG] retrieved bug, [FIX] retrieved fix*. The dense retriever is trained contrastively with each bug as query and its own fix as key, so no manual labels are needed.

→

STEP 3 · CODET5 GENERATOR

Ranked patch candidates

CodeT5-base (220M) decodes fixes with beam search (beam 5, or 100 on Defects4J). The top candidates are checked by exact match, a linter, or unit tests.

Leakage guard: during training the retriever may not return the true fix for the same bug; otherwise the model just learns to copy. The method needs no ASTs, so it works for any language.

Source: Wang, Wang, Joty, Hoi, ESEC/FSE 2023, Sections 3.1 to 3.4.

What the RAP-Gen results show

Retriever (TFix, exact match %)	EM
Random retrieved pair	52.98
No retriever (CodeT5 alone)	53.46
BM25 only (lexical)	53.88
CodeT5 dense only (semantic)	53.93
Hybrid BM25 + CodeT5 (RAP-Gen)	54.15

A random example is **worse than no example**: retrieval helps only when it is relevant.

54.15

TFix exact match vs 49.58 for T5-large, a model 3.5× larger: 478 more bugs fixed

78.80

TFix error-removal rate vs 69.30 for T5-large

24.80 / 15.84

Code Refinement exact match, small / medium (CodeT5: 21.61 / 13.96)

125

Defects4J bugs fixed with perfect localisation (72 + 53) vs 110 for SelfAPR

Source: RAP-Gen Tables 3, 6, 7, 9 and Figure 4. TFix = JavaScript linter bugs; Code Refinement and Defects4J = Java.

Building VibeApps and VibeVulns

VIBEAPPS · FINDING VIBE-CODED APPS

- **74,800** GitHub repos found by agent fingerprints: a `.claude/` folder (Claude Code) or Lovable's author meta tag.
- **38,078** kept after quality filters (README, size, commits, a real app).
- **9,041** classed as vibe-coded: first commit by AI, and at least 90% of commits and of lines written by AI.
- 96% are web apps (React, Vite, Tailwind; 55% use Supabase). **984** web apps have live deployments.

VIBEVULNS · AUDITING 200 DEPLOYED APPS

- **2 × 2 audit:** Claude Code and GitHub Copilot agents, each with two security skill sets: 800 reports, 11,150 raw findings.
- Deduplicated to 2,125, then a validation agent traced exploit paths with harmless probes: 1,287.
- Two security experts confirmed each one: **1,186** vulnerabilities, rated with OWASP (Cohen's κ = 0.87).
- Miss-rate checks: blind human audit (miss rate under 15%) and 95.7% recall on OWASP Juice Shop.

The apps are big and built fast: median **8,254 lines** of code written over a median of **8.39 days**, so there is little time for review before deployment.

Source: Deng, Fan, Meng, arXiv 2606.23130v4, Sections 3 and 4. Audit cost: 2.85B tokens, 128 compute hours.

Insecurity is the norm, not the exception

91.0%
of 200 deployed apps have at least one vulnerability (median 6 each)

65.77%
of 1,186 vulnerabilities rated Critical (18.47%) or High (47.30%)

52.7%
are in backend code; 35.9% frontend; 11.4% configuration

0.85→0.22
vulnerabilities per 1k lines, smallest to largest apps: small apps are not safer

OWASP category	Vulns	Apps affected
Broken access control	339 (28.6%)	64.0%
Injection	199 (16.8%)	54.0%
Authentication failures	176 (14.8%)	51.5%
Cryptographic failures	112 (9.4%)	36.5%
Security misconfiguration	108 (9.1%)	36.5%
Supply chain failures	73 (6.2%)	28.0%

The top three account for **60.2%** of all vulnerabilities: failures at trust boundaries. A real example: CVE-2025-48757 (CVSS 9.3), missing Supabase Row-Level Security in Lovable sites.

Source: arXiv 2606.23130v4, Section 5, Table 1, Table 2, Figure 2.

Eight failure modes, three agent defects

Defect class	Failure mode (stage)	What happens	Vulns
Memory 160 · 13.5%	Incomplete change propagation (iteration)	A new guard is added to new routes but not back-fitted to old ones, or a new path skips an existing guard	110 · 9.3%
	Forgotten obligations (deployment)	A `TODO: add password verification` is shipped as is	50 · 4.2%
Objective 274 · 23.1%	Demo-oriented design (specification)	Base64 "encryption", static demo keys, placeholder authorisation	174 · 14.7%
	Function-fix side effects (iteration)	An error is "fixed" by weakening security, e.g. a hard-coded login bypass when auth breaks	100 · 8.4%
Knowledge 752 · 63.4%	Hidden security rules (implementation)	Unstated rules are skipped: hash passwords, sanitise input, check on the server	658 · 55.5%
	User-dependent security (deployment)	Agent expects the user to gitignore secrets or set env vars; the user does not	72 · 6.1%
	Hallucination (implementation)	Calls a non-existent API (`crypto.createCipherGCM`); data stays unencrypted	13 · 1.1%
	Insecure instructions (specification)	CLAUDE.md asks to "protect" an API key with a Caesar cipher, and the agent complies	9 · 0.8%

Source: arXiv 2606.23130v4, Section 6, Figure 3. Open coding by two authors, Cohen's κ = 0.82.

PAPER 02 · RQ4 · MITIGATION

What reduces vulnerabilities, and what does not

Replay of 70 real vulnerabilities, 8 configurations, 3 runs each: **1,680 runs**. One factor changed at a time.

Configuration	Reintroduced	vs base
Baseline (GPT-5.6-Terra, short prompt)	25.7%	—
Prompt adds "ready for production"	11.0%	−14.8 pp
Harness with security-hardening skill	11.9%	−13.8 pp
"Polished" / "self-check" prompt	22.9 / 23.8%	−2.9 / −1.9
Stronger model (Sol)	26.2%	+0.5 pp
Weaker model (Luna)	39.5%	+13.8 pp
Detailed "professional" prompt	45.7%	+20.0 pp

No setting removes the risk

35 vulnerabilities came back under 3 or more configurations. Nothing eliminated all of them.

Results vary between identical runs

22.0% of vulnerability-configuration cells gave different outcomes across three identical repeats.

Awareness-action gap

In 90 of 434 insecure runs (20.7%) the agent named the risk, then shipped insecure code with only a warning or comment.

Source: arXiv 2606.23130v4, Section 7, Tables 3 and 4.

COMPARISON

The two papers side by side

Aspect	RAP-Gen	(In)Security of Vibe-Coded Apps
Contribution	Repair framework: hybrid retriever + CodeT5 generator	Empirical study + two datasets (VibeApps, VibeVulns)
Question	Does a retrieved past fix help a model write a correct patch?	How insecure are real vibe-coded apps, why, and what helps?
Data	TFix (JavaScript), Code Refinement and Defects4J (Java)	9,041 apps; 1,186 human-confirmed vulnerabilities in 200 deployed apps
How correctness is judged	Exact match, linter error removal, unit tests + manual check	Agent audit, exploit probe, two human experts (κ = 0.87)
Headline result	54.15% EM on TFix; 125 Defects4J bugs with perfect localisation	91% of apps vulnerable; 65.77% Critical or High
Main limit	Small model era; exact match is strict; Code Refinement pairs noisy; Java and JS only	Only two agents and public GitHub apps; auditor overlaps with an audited agent

Compiled from the two papers' reported results.

CLOSING

Takeaways and what they mean for RepoPilot

RAP-GEN

Takeaway: a relevant past fix guides repair; combining word-based and meaning-based search is the most robust.

For us: supports a retrieval-backed pattern library for bounded repairs. A random example scored below no example, so we must measure retrieval quality and keep benchmark apps out of the library.

VIBE-CODED SECURITY

Takeaway: nine in ten deployed apps are vulnerable, mostly through missing access control, injection and auth checks.

For us: confirms our stack (React/Vite/Supabase) and check families. "Function-fix side effects" is why a patch that weakens a check is not a fix. TODOs and CLAUDE.md are useful intent evidence. Run-to-run variance means baselines need repeated runs.

The "For us" lines are this summary's reading, not claims made by the papers.